DBMS LEVEL 5 EXAM SOLUTIONS

Comprehensive Study & Revision Guide | All 150 Questions Solved

Module 1: Problem Solving Workplace Safety and Database Modeling

Theory Questions

1. A sales report repeatedly shows an incorrect total. Distinguish the problem, the symptom, and possible root causes. How would you measure the expected outcome after correction?

Answer:

- **Symptom:** The incorrect total displayed on the final sales report output.
- **Problem:** Data inaccuracy in the sales reporting calculation pipeline.
- **Possible Root Causes:** Incorrect SQL aggregation query (e.g., missing `WHERE` clause or duplicate `JOIN` rows), corrupted database entries, or floating-point rounding errors.
- **Measurement:** Re-run the report against audited manual receipts; 100% match across 0 error variance validates correction.

2. A cable lies across a walkway in a server room. Explain the difference between a hazard, a risk, and an incident in this scenario.

Answer:

- **Hazard:** The trailing cable itself (a physical object with potential to cause harm).
- **Risk:** The probability and severity of a technician tripping over the cable and sustaining an injury.
- **Incident:** The actual event where a person trips over the cable, regardless of whether injury occurs.

3. How would you choose PPE for hardware servicing and for work in a software lab? Explain how the tasks and risks affect your choices.

Answer:

- **Hardware Servicing:** Anti-static (ESD) wrist strap, safety glasses, and steel-toe/anti-slip footwear. Protects against electrostatic discharge, flying component debris, and heavy falling equipment.
- **Software Lab:** Ergonomic wrist rests and blue-light filtering glasses. Addresses lower physical risks like repetitive strain injury (RSI) and digital eye strain during prolonged screen exposure.

4. A customer can have multiple phone numbers, and age is calculated from date of birth. Classify both attributes and explain how you would store or derive them.

Answer:

- **Phone Numbers:** Multivalued attribute. Stored in a separate child table (`CustomerPhone`) with a Foreign Key referencing `CustomerID` .
- **Age:** Derived attribute. Do not store in DB to prevent staleness; calculate dynamically in queries using `DATEDIFF(CURRENT_DATE, DateOfBirth)` .

5. Give two reasons for choosing a relational model for an organization's employee records and related sales records.

Answer:

1. **ACID Compliance & Referential Integrity:** Enforces Foreign Keys to prevent orphan sales records and ensure data consistency.
2. **Declarative Query Flexibility:** Enables complex relational joins and aggregations across employees and sales data without data redundancy.

6. A team blames new software for a reporting error without evidence. What bias might be affecting its judgment? What would you verify before choosing a corrective action?

Answer:

- **Bias:** Confirmation bias / Availability heuristic (blaming recent system changes without empirical proof).
- **Verification:** Audit error logs, compare baseline data inputs, check calculation scripts, and reproduce the error in a sandbox environment using pre-upgrade data.

7. A problem affects both the software and support teams. How would a functional structure and a matrix structure differ in assigning responsibility and making decisions?

Answer:

- **Functional Structure:** Vertically siloed. Each department manager addresses their own side; decision-making is slow as escalation passes up separate chains.
- **Matrix Structure:** Project/Incident lead coordinates cross-functional members from both teams. Decisions are joint, rapid, and shared across horizontal project goals.

8. Lab workers report increasing eye strain and wrist pain after long sessions. Explain the risk factors, preventive measures, and how to evaluate their effectiveness.

Answer:

- **Risk Factors:** Poor lighting/glare, unergonomic seat/desk height, continuous screen focus, repetitive mouse/keyboard motion.
- **Preventive Measures:** Implement 20-20-20 rule breaks, adjustable ergonomic chairs/monitor stands, anti-glare screens, ergonomic keyboards.
- **Evaluation:** Conduct post-implementation health surveys and track medical incident/complaint rates over 30 days.

9. An OrderLine can be identified by LineNo only within its Order. Explain why it is a weak entity and identify a suitable key.

Answer:

- **Why Weak:** `OrderLine` cannot be uniquely identified by its partial key (`LineNo`) alone without its owner entity (`Order`).
- **Suitable Key:** Composite Primary Key `(OrderID, LineNo)` , where `OrderID` is a Foreign Key referencing `Orders` .

10. Student-Course is a many-to-many relationship and Student-IDCard is one-to-one. Explain how to enforce both relationships in a relational schema.

Answer:

- **Student-Course (M:N):** Create a junction table `Enrollment(StudentID, CourseID)` with a composite Primary Key `(StudentID, CourseID)` .
- **Student-IDCard (1:1):** Place `StudentID` as a Foreign Key in `IDCard` with a `UNIQUE` constraint, or share the exact same Primary Key across both tables.

Practical Questions

11. In one month, there were 18 input errors, 7 report filter errors, 4 connection failures, and 1 permission error. Create a Pareto chart and identify which problems to address first.

Answer:

Total errors = 30. Calculations:

Error Type	Count	%	Cumulative %
Input Errors	18	60.0%	60.0%
Report Filter Errors	7	23.3%	83.3%
Connection Failures	4	13.3%	96.6%
Permission Errors	1	3.3%	100.0%

Priority: Address **Input Errors** and **Report Filter Errors** first, as they account for 83.3% of all issues (violating the 80/20 threshold).

12. Create a database lab inspection checklist for cables, posture, lighting, cleanliness, and safety signs. Record sample findings.

Answer:

Category	Inspection Item	Status	Sample Findings
Cables	Trunking & Routing	FAIL	Loose network cable across Aisle 2 walkway.
Posture	Ergonomic Chair Adjustments	PASS	Chairs calibrated to standard desk heights.
Lighting	Luminance & Glare Control	PASS	Diffused LED panels active; no monitor reflections.
Cleanliness	Server Vent Dust Levels	FAIL	Dust accumulation on intake fans of Rack B.
Safety Signs	Emergency Exit Signage	PASS	Exit signs lit and clear of obstruction.

13. A person trips over a loose cable in the lab and suffers a minor injury. Write an incident report covering the event, immediate action, responsible person, and preventive measures.

Answer:

- **Event:** Student tripped over loose patch cable in Lab 3 walkway at 14:15. Resulted in minor knee scrape.
- **Immediate Action:** Administered first-aid kit, secured hazard zone with safety cones.
- **Responsible Person:** Lab Safety Officer (John Doe).
- **Preventive Measures:** Installed cable bridges immediately; scheduled full cable trunking audit for all lab walkways by Friday.

14. Draw an ER diagram for Customer, Order, and Orderline. Each order belongs to one customer and has one or more lines. Mark the keys and cardinalities.

Answer:

```
[ Customer ] (1) ------ (0..N) [ Order ] (1) ------ (1..N) [[ OrderLine ]]
  PK: CustomerID                 PK: OrderID                 PK: (OrderID, LineNo)
                                 FK: CustomerID              FK: OrderID
```

15. Build a data dictionary for Product covering SKU, name, price, active status, and date added. Specify the data type, required status, and constraints for each field.

Answer:

Field Name	Data Type	Required	Constraints
SKU	VARCHAR(30)	Yes (NOT NULL)	PRIMARY KEY
Name	VARCHAR(100)	Yes (NOT NULL)	None
Price	DECIMAL(10,2)	Yes (NOT NULL)	CHECK (Price >= 0)
ActiveStatus	BOOLEAN	Yes (NOT NULL)	DEFAULT TRUE

Field Name	Data Type	Required	Constraints
DateAdded	DATE	Yes (NOT NULL)	DEFAULT CURRENT_DATE

16. A member can use multiple phone numbers. Write the Member and MemberPhone schemas and specify the constraint that prevents entering the same number twice for one member.

Answer:

```
CREATE TABLE Member (
    MemberID INT PRIMARY KEY,
    MemberName VARCHAR(100) NOT NULL
);

CREATE TABLE MemberPhone (
    MemberID INT NOT NULL,
    PhoneNumber VARCHAR(20) NOT NULL,
    PRIMARY KEY (MemberID, PhoneNumber),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID) ON DELETE CASCADE
);
```

17. Model Department and Employee in an IDE or modeling tool. Every employee must belong to a department. Generate the DDL and verify the keys against the model.

Answer:

```
CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL
);

CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    EmpName VARCHAR(100) NOT NULL,
    DepartmentID INT NOT NULL,
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

Verification: PKs enforce entity uniqueness; `NOT NULL` on `DepartmentID` guarantees mandatory department assignment.

18. Build a cause-and-effect diagram for inflated order totals using duplicate rows, incorrect joins, and incorrect unit prices as possible causes. Give a specific test for each cause.

Answer:

- **Problem:** Inflated Order Totals
- **Cause 1: Duplicate Rows:** Test: `SELECT OrderID, ProductID, COUNT(*) FROM OrderLine GROUP BY OrderID, ProductID HAVING COUNT(*) > 1;`
- **Cause 2: Incorrect Joins (Cartesian):** Test: `SELECT OrderID, SUM(UnitPrice * Quantity) FROM OrderLine GROUP BY OrderID;` compare with `Orders.TotalAmount`.
- **Cause 3: Incorrect Unit Prices:** Test: `SELECT ol.LineNo, ol.UnitPrice, p.Price FROM OrderLine ol JOIN Product p ON ol.ProductID = p.ProductID WHERE ol.UnitPrice != p.Price;`

19. The same error occurred 12, 5, and 8 times over three weeks. Prepare an action plan with ownership, deadlines, success criteria, and a review date.

Answer:

Action Item	Owner	Deadline	Success Criteria	Review Date
Fix DB validation script	DBA Lead	7 Days	0 instances of error in logs	Week 4 (Mon)
Retrain entry staff	QA Manager	10 Days	100% staff pass re-test	Week 4 (Wed)

20. Plan a drill for smoke coming from electrical equipment in a lab. Demonstrate reporting, evacuation, assembly, and incident recording according to workplace procedures.

Answer:

1. **Reporting:** Immediately trigger nearest fire alarm call point; inform Security/Emergency Services.
2. **Evacuation:** Shut main power breaker if safe; evacuate lab calmly using marked emergency exits. Do NOT use elevators.
3. **Assembly:** Muster at Assembly Point A; roll-call performed by Safety Lead.
4. **Recording:** Log incident timeline, response duration, and root cause in Safety Register within 2 hours.

21. A legacy results table contains StudentID, StudentName, CourseID, CourseName, and Marks. The StudentID-CourseID pair is unique. Create an ER diagram and relational schema that reduce redundancy.

Answer:

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL
);

CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL
);

CREATE TABLE ExamResult (
    StudentID INT,
    CourseID INT,
    Marks DECIMAL(5,2) NOT NULL,
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

22. An online store can change Product Price, while existing orders must retain the original selling price. Design the Product, Order, and OrderLine schemas and write the DDL.

Answer:

```
CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    Price DECIMAL(10,2) NOT NULL
);

CREATE TABLE OrderLine (
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    SellingPrice DECIMAL(10,2) NOT NULL, -- Snapshotted price at order time
    Quantity INT NOT NULL,
    PRIMARY KEY (OrderID, ProductID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
```

23. A library can have several copies of a book, and a member may borrow the same copy on different occasions. Draw an ER diagram for Book, Copy, Member, and Loan and show the key that distinguishes each loan.

Answer:

Key for Loan: `LoanID` (Surrogate PK) OR Composite Key `(BookID, CopyNo, MemberID, BorrowDate)`.

```
[Book] (1)--(0..N) [Copy] (1)--(0..N) [Loan] (0..N)--(1) [Member]
PK: BookID         PK: (BookID, CopyNo)  PK: LoanID        PK: MemberID
```

24. In a TrainingCenter database, a course can have several batches, each with several students. Generate DDL from an ER diagram and write test cases for valid and invalid enrollments.

Answer:

```
CREATE TABLE Enrollment (
    BatchID INT NOT NULL,
    StudentID INT NOT NULL,
    EnrollmentDate DATE NOT NULL,
    PRIMARY KEY (BatchID, StudentID)
);

-- Test Valid:
INSERT INTO Enrollment VALUES (101, 5001, CURRENT_DATE); -- Succeeds
-- Test Invalid (Duplicate Enrollment):
INSERT INTO Enrollment VALUES (101, 5001, CURRENT_DATE); -- Fails: Primary Key Violation
```

25. Prepare a handover sheet for the Training Center model. Record relationships, constraints, data types, and design decisions, and show how to locate the revised version later.

Answer:

- **Model Version:** v2.4 (TrainingCenter_DB)
- **Key Relationships:** Course (1:N) Batch; Batch (M:N) Student via Enrollment.
- **Core Constraints:** Composite PK on `Enrollment(BatchID, StudentID)`; FK cascade on delete.
- **Repository Path:** `https://git.company.internal/db/training-center/tree/v2.4`

Module 2: Database Setup and Object Management

Theory Questions

26. How would workload, memory, and storage guide your choice of database environment for an 8 GB training lab and for an office server with multiple concurrent users?

Answer:

- **8 GB Lab:** Light workload, constrained RAM. Choose lightweight/containerized DBMS (e.g., PostgreSQL / SQLite) with low memory buffer allocations (e.g., 1-2 GB max memory target).
- **Office Server:** Heavy concurrent workload. Requires multi-core CPU, 32GB+ RAM, fast SSD/NVMe RAID storage, and enterprise DBMS (e.g., SQL Server / Oracle / PostgreSQL Enterprise) tuned for high concurrency connection pooling.

27. A running database server produces a client connection timeout. In what order would you check the host, port, listener, and firewall? Explain your reasoning.

Answer:

1. **Host:** Ping server IP to verify basic network connectivity.
2. **Firewall:** Check if network/host firewall blocks traffic on DB port.
3. **Listener:** Verify if DB service/listener process is running and bound to socket.
4. **Port:** Check if client configuration points to the correct TCP port number.

28. Explain why DECIMAL is suitable for UnitPrice and why FLOAT may suit an approximate experimental measurement.

Answer:

- **DECIMAL:** Exact fixed-point numeric representation; eliminates binary floating-point rounding errors crucial for financial currency accuracy.
- **FLOAT:** Approximate hardware-accelerated floating-point representation; ideal for scientific metrics with high range where tiny precision variations are acceptable.

29. Why might Employee Number need a PRIMARY KEY while Email needs separate UNIQUE and NOT NULL constraints?

Answer:

Employee Number serves as the immutable surrogate primary join key for internal entity integrity. Email is a business attribute that can change over time; applying `UNIQUE` and `NOT NULL` guarantees non-duplicate, required data while keeping foreign key relationships stable on Employee Number.

30. A report can use both a view and an index. Explain what each contributes and where the index incurs maintenance overhead.

Answer:

- **View:** Provides logical query abstraction and simplifies complex join syntaxes.
- **Index:** Speeds up data retrieval performance via physical index lookup structures (B-Trees).
- **Maintenance Overhead:** Every `INSERT`, `UPDATE`, and `DELETE` forces the engine to synchronously update the index structure, writing extra disk I/O.

31. A credit limit must be validated before any change. Which of a function, stored procedure, and trigger would you choose to enforce this automatically? Explain an appropriate role for each of the other two.

Answer:

- **Trigger (BEFORE UPDATE):** Chosen for automatic, transparent enforcement on all data changes.
- **Function:** Calculates and returns a scalar boolean state (e.g., `fn_IsCreditValid(ID, limit)`).
- **Stored Procedure:** Encapsulates complete business workflows (e.g., `sp_UpdateCustomerCredit`).

32. A view and a procedure reference a column that must change. How would you check dependencies before the change and verify integrity afterward?

Answer:

- **Pre-Check:** Query system catalog dependency views (e.g., `information_schema.routine_column_usage` or `sys.sql_expression_dependencies`).
- **Post-Verification:** Recompile objects and run automated unit test queries against both view and procedure to confirm zero runtime errors.

33. Existing records contain duplicates. What must be checked, corrected, and approved before adding a UNIQUE constraint?

Answer:

1. Identify duplicate key groups using `GROUP BY ... HAVING COUNT(*) > 1`.
2. Deduplicate records (delete/merge invalid rows).
3. Obtain formal sign-off from data owner prior to applying the `ALTER TABLE ADD CONSTRAINT UNIQUE` statement.

34. A migration script succeeds in a test environment and fails in the target environment. How would you isolate the cause by checking the schema, dependencies, and privileges?

Answer:

- **Schema Drift:** Compare DDL objects across environments.
- **Dependencies:** Verify presence of required referenced tables/views in target.
- **Privileges:** Verify target DB user possesses necessary DDL rights (e.g., `ALTER`, `CREATE TABLE`).

35. A new row receives a default status. Explain how DEFAULT, CHECK, and NOT NULL work together when an invalid status is supplied, NULL is supplied, or the value is omitted.

Answer:

- **Invalid Status Supplied:** Rejected by `CHECK` constraint.
- **NULL Supplied:** Rejected by `NOT NULL` constraint (if `DEFAULT` doesn't override explicit NULLs).
- **Value Omitted:** `DEFAULT` constraint injects standard initial value, which passes `NOT NULL` and `CHECK`.

Practical Questions

36. Prepare a prerequisite sheet for a selected DBMS covering OS, edition, RAM, storage, and client compatibility.

Answer:

- **DBMS:** PostgreSQL 16 Community Edition
- **OS:** Ubuntu 22.04 LTS / Windows Server 2022
- **RAM:** 4 GB Minimum (8 GB Recommended)
- **Storage:** 20 GB SSD SSD space available
- **Client Compatibility:** pgAdmin 4 v8.0+, psql CLI

37. Install a database server and compatible client in a lab or VM. Verify the service, port, and test connection, then record the setup.

Answer:

```
 # Service & Port Verification commands
sudo systemctl status postgresql
netstat -tuln | grep 5432
psql -h localhost -U postgres -c "\conninfo"
```

Outcome: Connection established successfully on port 5432.

38. Reproduce a connection error by entering the wrong client port. Find the correct port, restore the connection, and document both outcomes.

Answer:

- **Attempt 1 (Port 5433):** Error: `Connection refused (0x00020061)`.
- **Resolution:** Identified listening port 5432 via `netstat`. Re-configured client target.
- **Attempt 2 (Port 5432):** Connected successfully.

39. Create Customer and Orders tables with a primary key, foreign key, NOT NULL, and a CHECK for a nonnegative total. Test valid and invalid inserts.

Answer:

```
 CREATE TABLE Customer (CustomerID INT PRIMARY KEY, Name VARCHAR(50) NOT NULL);
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT NOT NULL REFERENCES Customer(CustomerID),
    Total DECIMAL(10,2) CHECK (Total >= 0)
);
-- Valid Insert:
INSERT INTO Customer VALUES (1, 'Alice'); INSERT INTO Orders VALUES (101, 1, 50.00);
-- Invalid Insert (Negative Total):
INSERT INTO Orders VALUES (102, 1, -10.00); -- Fails CHECK constraint
```

40. Create a Product table with unique SKU, nonnegative price, and active status defaulting to true. Test relevant columns with an explicit NULL and an omitted value.

Answer:

```
CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    SKU VARCHAR(20) UNIQUE NOT NULL,
    Price DECIMAL(10,2) CHECK (Price >= 0),
    IsActive BOOLEAN DEFAULT TRUE
);
-- Omitted value test (IsActive defaults to TRUE):
INSERT INTO Product (ProductID, SKU, Price) VALUES (1, 'SKU101', 15.00);
-- Explicit NULL test (fails NOT NULL):
INSERT INTO Product VALUES (2, NULL, 20.00, TRUE);
```

41. Create a view on Orders(OrderID, CustomerID, OrderDate, Total) that shows order date and total. Compare the output with the base table.

Answer:

```
CREATE VIEW v_OrderSummary AS
SELECT OrderDate, Total FROM Orders;

SELECT * FROM v_OrderSummary;
-- Output matches subset of Orders table, suppressing OrderID and CustomerID columns.
```

42. Orders is frequently filtered on CustomerID and OrderDate. Create a suitable index and verify the column order in the database object listing.

Answer:

```
CREATE INDEX idx_orders_cust_date ON Orders(CustomerID, OrderDate);
-- Object listing query:
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders';
```

43. Write a function that returns a customer's total order value from a CustomerID. Define and test expected behavior for a customer with orders, one without orders, and an unknown ID.

Answer:

```
CREATE FUNCTION fn_GetCustomerTotal(p_cust_id INT)
RETURNS DECIMAL(10,2) AS $$
    SELECT COALESCE(SUM(Total), 0.00) FROM Orders WHERE CustomerID = p_cust_id;
$$ LANGUAGE SQL;

-- Test Case 1 (Has orders): Returns 150.00
-- Test Case 2 (No orders / Unknown ID): Returns 0.00
```

44. Write a stored procedure that lists orders for a given CustomerID and date range. Test an empty result and a range with the start date after the end date.

Answer:

```
 CREATE PROCEDURE sp_GetOrdersByRange(p_cid INT, p_start DATE, p_end DATE)
LANGUAGE plpgsql AS $$
BEGIN
    IF p_start > p_end THEN
        RAISE EXCEPTION 'Start date cannot be after End date';
    END IF;
    SELECT * FROM Orders WHERE CustomerID = p_cid AND OrderDate BETWEEN p_start AND p_end;
END; $$;
```

45. Create a trigger that records OrderID, old status, new status, and change time in OrderStatusLog when an order's status changes. Also test an update that leaves the status unchanged.

Answer:

```
 CREATE TRIGGER trg_log_status_change
BEFORE UPDATE ON Orders
FOR EACH ROW
WHEN (OLD.Status IS DISTINCT FROM NEW.Status)
EXECUTE FUNCTION log_status_change();
-- Unchanged updates bypass execution due to WHEN clause.
```

46. Insert a test row using one of two client sessions. Check whether the second session can see the row before and after COMMIT. Record the isolation level and autocommit settings.

Answer:

- **Isolation Level:** Read Committed | **Autocommit:** OFF.
- **Before COMMIT:** Session 2 queries return 0 rows (Uncommitted changes isolated).
- **After COMMIT:** Session 2 queries see the new row instantly.

47. Write a migration script that makes Email mandatory and unique in an existing Customer table. Detect NULLs and duplicates and verify the change in a test database.

Answer:

```
 -- Detect bad data:
SELECT Email FROM Customer WHERE Email IS NULL OR Email IN
  (SELECT Email FROM Customer GROUP BY Email HAVING COUNT(*) > 1);

-- Clean data and alter schema:
UPDATE Customer SET Email = 'missing_' || CustomerID || '@domain.com' WHERE Email IS NULL;
ALTER TABLE Customer ADD CONSTRAINT unique_email UNIQUE(Email);
ALTER TABLE Customer ALTER COLUMN Email SET NOT NULL;
```

48. Identify the view and procedure dependencies before renaming Orders.Total. Execute the revised objects and document their outputs and the migration order.

Answer:

1. Drop dependent views/procedures.
2. Rename column: `ALTER TABLE Orders RENAME COLUMN Total TO TotalAmount;`
3. Recreate views/procedures pointing to `TotalAmount`.
4. Execute unit tests to verify zero failure.

49. An obsolete view is approved for retirement. Check dependencies, verify the replacement view's output, and write the script to remove the old view safely.

Answer:

```
-- Confirm no dependencies exist:
SELECT * FROM information_schema.view_table_usage WHERE view_name = 'v_old_orders';
-- Drop safely:
DROP VIEW IF EXISTS v_old_orders RESTRICT;
```

50. Prepare an object inventory for a small order database containing primary keys, foreign keys, a view, a function, and a trigger. Record each object's purpose, dependencies, and one validation result.

Answer:

Object Name	Type	Purpose	Dependencies	Validation Result
pk_orders	PK Constraint	Uniqueness of OrderID	Orders table	PASS (Rejects duplicates)
fk_customer	FK Constraint	Referential integrity	Customer table	PASS (Rejects invalid IDs)
v_OrderSummary	View	Reporting abstraction	Orders table	PASS (Returns 100 rows)
fn_GetTotal	Function	Calc customer spending	Orders table	PASS (Accurate sums)
trg_log_status	Trigger	Audit status edits	OrderStatusLog	PASS (Logs state edits)

Module 3: Querying and Modifying Data with SQL

Theory Questions

51. A report must include customers who have never ordered. Use an example to explain how INNER JOIN and LEFT JOIN differ.

Answer:

- `INNER JOIN`: Returns only rows with matching records in both tables (excludes customers with 0 orders).
- `LEFT JOIN`: Returns all rows from the left table (Customers), filling right table (Orders) columns with `NULL` for non-matching rows, preserving customers without orders.

52. A product report filters by price range, category, and active status. How can missing parentheses in a condition with AND and OR cause incorrect results?

Answer:

SQL evaluates `AND` prior to `OR`. Writing `WHERE Price > 100 AND Category = 'Tech' OR Active = true` evaluates as `(Price > 100 AND Category = 'Tech') OR (Active = true)`, returning non-tech products that are active regardless of price.

53. Ten sales rows include some NULL discounts. Explain what COUNT(*), COUNT(discount), and AVG(discount) calculate.

Answer:

- `COUNT(*)`: Counts total rows (returns 10).
- `COUNT(discount)`: Counts non-NULL discount values only.
- `AVG(discount)`: Calculates mean across non-NULL discount values (ignores NULL rows completely).

54. How would UNION and UNION ALL differ when combining customer lists from two branches?

Answer:

- `UNION`: Combines lists and performs duplicate elimination (slower).
- `UNION ALL`: Combines lists while retaining all duplicates (faster execution).

55. To find customers whose combined order value is at least 5,000, explain what WHERE and HAVING can filter and at what stage.

Answer:

- `WHERE`: Filters individual raw rows *before* aggregation.
- `HAVING`: Filters aggregated summary data (e.g., `HAVING SUM(Total) >= 5000`) *after* `GROUP BY`.

56. Why can a date condition on the right table in WHERE remove customers from a LEFT JOIN result? Explain how placing it in ON changes the result.

Answer:

A `WHERE` clause condition evaluates `NULL` right-table rows as false, converting the `LEFT JOIN` into an `INNER JOIN`. Moving the predicate into the `ON` clause evaluates the condition during join formation, preserving non-matching left rows with `NULL` fillers.

57. An order is inserted, but its stock update fails. Which ACID property applies? What can happen when autocommit is on?

Answer:

- **ACID Property:** Atomicity ("All-or-Nothing").
- **Autocommit Effect:** If enabled without explicit transactions, the `INSERT` commits permanently while stock update fails, leaving data in an inconsistent orphan state.

58. Joining Customers to Orders and OrderLine inflates the order count per customer. Explain the cause using join cardinality and show how to correct the count.

Answer:

- **Cause:** 1-to-Many join fan-out multiplies order rows by the number of matching line items.
- **Correction:** Use distinct counting: `COUNT(DISTINCT Orders.OrderID)`.

59. An operator writes an overly broad WHERE clause for a DELETE on Orders. Explain how previewing the rows, checking the affected row count, and rolling back can prevent an incorrect change.

Answer:

1. Preview target rows using `SELECT * FROM Orders WHERE ...` prior to deletion.
2. Execute deletion inside a explicit transaction block: `BEGIN TRANSACTION; DELETE ...;`
3. Inspect row count output; execute `ROLLBACK` if count is higher than expected.

60. Compare the result shape of a scalar subquery for average salary with that of an IN subquery for selected department IDs. What happens if the scalar subquery returns multiple rows?

Answer:

- **Scalar Subquery:** Returns a single value (1 row, 1 column).
- **IN Subquery:** Returns a 1-column list of multiple values.
- **Multiple rows error:** Scalar subqueries fail at runtime with a cardinal error (e.g., "Subquery returned more than 1 row").

Practical Questions

61. From Orders (OrderID, CustomerID, Status, OrderDate, Total), list completed orders in 2026. Sort by Total descending and then OrderID ascending, and return the first five rows.

Answer:

```
SELECT * FROM Orders
WHERE Status = 'Completed'
  AND OrderDate BETWEEN '2026-01-01' AND '2026-12-31'
ORDER BY Total DESC, OrderID ASC
LIMIT 5;
```

62. From Product(ProductID, Name, Category, Price), find names beginning with 'A' and prices from 500 to 2,000. Display names using UPPER.

Answer:

```
SELECT UPPER(Name) AS UPPER_NAME, Price
FROM Product
WHERE Name LIKE 'A%' AND Price BETWEEN 500 AND 2000;
```

63. Using Customer(CustomerID, Name) and Orders(OrderID, CustomerID, Total), return every customer's name and order count. Show zero for customers without orders.

Answer:

```
SELECT c.Name, COUNT(o.OrderID) AS OrderCount
FROM Customer c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name;
```

64. Using Employee (EmployeeID, Name, ManagerID), write a self join that shows each employee and their manager. Include employees without a manager.

Answer:

```
SELECT e.Name AS EmployeeName, m.Name AS ManagerName
FROM Employee e
LEFT JOIN Employee m ON e.ManagerID = m.EmployeeID;
```

65. From Orders(OrderID, Total, Discount), calculate net amount treating a NULL discount as zero. Use CASE to label net amounts of at least 5,000 as 'High'.

Answer:

```
SELECT OrderID,
       (Total - COALESCE(Discount, 0)) AS NetAmount,
       CASE WHEN (Total - COALESCE(Discount, 0)) >= 5000 THEN 'High'
            ELSE 'Standard' END AS Priority
FROM Orders;
```

66. From Product(ProductID, Category, Price), show the product count and average price for each category with at least three products.

Answer:

```
SELECT Category, COUNT(*) AS ProductCount, AVG(Price) AS AvgPrice
FROM Product
GROUP BY Category
HAVING COUNT(*) >= 3;
```

67. Use a simple subquery on Employee (EmployeeID, DepartmentID, Salary) to find employees earning more than the organization-wide average.

Answer:

```
SELECT EmployeeID, Salary
FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

68. Populate BranchA(CustomerID) and BranchB(CustomerID) with duplicate and overlapping IDs, excluding NULLs. Use two set operations to find IDs in both branches and IDs only in BranchA.

Answer:

```
-- IDs in both branches:
SELECT CustomerID FROM BranchA INTERSECT SELECT CustomerID FROM BranchB;
-- IDs only in BranchA:
SELECT CustomerID FROM BranchA EXCEPT SELECT CustomerID FROM BranchB;
```

69. Find customers who ordered during a specified month in Orders (OrderID, CustomerID, OrderDate, Total). Use an IN subquery to show their names and email addresses from Customers.

Answer:

```
SELECT Name, Email
FROM Customers
WHERE CustomerID IN (
    SELECT CustomerID FROM Orders
    WHERE OrderDate BETWEEN '2026-05-01' AND '2026-05-31'
);
```

70. From Sales (SaleID, Region, SaleDate, Units, UnitPrice), filter by date first, then calculate total revenue and average line revenue by region. Verify boundary dates.

Answer:

```
SELECT Region,
       SUM(Units * UnitPrice) AS TotalRevenue,
       AVG(Units * UnitPrice) AS AvgLineRevenue
FROM Sales
WHERE SaleDate >= '2026-01-01' AND SaleDate <= '2026-12-31'
GROUP BY Region;
```

71. Use INSERT... SELECT to copy canceled orders from 2025 from Orders (OrderID, Status, OrderDate, Total) into ArchiveOrders. Specify the target columns and compare source and target row counts.

Answer:

```
 INSERT INTO ArchiveOrders (OrderID, Status, OrderDate, Total)
SELECT OrderID, Status, OrderDate, Total
FROM Orders
WHERE Status = 'Canceled' AND OrderDate BETWEEN '2025-01-01' AND '2025-12-31';

-- Compare Counts:
SELECT COUNT(*) FROM Orders WHERE Status = 'Canceled' AND OrderDate BETWEEN '2025-01-01'
AND '2025-12-31';
SELECT COUNT(*) FROM ArchiveOrders;
```

72. Increase Stock by 5 only for active products with Stock below 10 in Inventory(ProductID, Stock, Active). Preview the rows, check affected rows, and verify the result before COMMIT.

Answer:

```
 BEGIN;
-- Preview:
SELECT * FROM Inventory WHERE Active = true AND Stock < 10;
-- Update:
UPDATE Inventory SET Stock = Stock + 5 WHERE Active = true AND Stock < 10;
-- Verify & Commit:
SELECT * FROM Inventory WHERE Active = true AND Stock < 15;
COMMIT;
```

73. Delete rejected rows before a specified cutoff from ReturnRequests (RequestID, Status, RequestDate). Roll back a test run and verify that the original count returns.

Answer:

```
 BEGIN;
DELETE FROM ReturnRequests WHERE Status = 'Rejected' AND RequestDate < '2026-01-01';
-- Rollback test:
ROLLBACK;
-- Original count maintained.
```

74. Write the two UPDATE statements to transfer 1,000 from a source account to a destination account in Accounts (AccountID, Balance). Demonstrate COMMIT for success and ROLLBACK for an invalid destination.

Answer:

```
 BEGIN;
UPDATE Accounts SET Balance = Balance - 1000 WHERE AccountID = 101;
UPDATE Accounts SET Balance = Balance + 1000 WHERE AccountID = 202;
-- Verify destination validity: If valid THEN COMMIT ELSE ROLLBACK;
COMMIT;
```

75. Test a multi-row INSERT into OrderLine (OrderID, ProductID, Quantity) using valid data, a missing foreign key reference, and an invalid quantity. Use the error messages to correct the data and record the result.

Answer:

```
-- Attempt (Fails due to FK/CHECK violations):
INSERT INTO OrderLine VALUES (1, 99999, 2), (1, 101, -5);
-- Correction applied:
INSERT INTO OrderLine VALUES (1, 201, 2), (1, 101, 5); -- Success
```

Module 4: Advanced SQL and Query Optimization

Theory Questions

76. A report first calculates sales per customer and then compares results by region. If you split the query into multiple CTEs, what should each step return?

Answer:

- **CTE 1 (CustomerTotals):** Returns `CustomerID`, `RegionID`, and aggregated `TotalSales`.
- **CTE 2 (RegionAverages):** Takes output of CTE 1 and returns `RegionID` alongside `AvgRegionSales`.

77. Explain the roles of the anchor and recursive parts of a recursive CTE for an employee-manager hierarchy. When does recursion stop?

Answer:

- **Anchor Member:** Establishes base set (e.g., top-level CEO/Manager with `ManagerID IS NULL`).
- **Recursive Member:** Joins anchor results iteratively to subordinates.
- **Termination:** Stops automatically when the recursive step returns an empty result set.

78. How do ROW_NUMBER, RANK, and DENSE_RANK handle tied sales totals? Illustrate using 100, 100, and 80.

Answer:

Sales Total	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100	2	1	1
80	3	3 (Skips 2)	2 (No skip)

79. How do the window frames differ for a monthly running total and a three-month moving average that includes the current month?

Answer:

- **Monthly Running Total:** `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`
- **Three-Month Moving Avg:** `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`

80. Explain how blocking and deadlock differ when two sessions update records. Give an example of each.

Answer:

- **Blocking:** Session A locks Row 1; Session B attempts edit on Row 1 and waits for A to commit.
- **Deadlock:** Session A locks Row 1 and requests Row 2; Session B locks Row 2 and requests Row 1. System aborts one transaction.

81. Running a staging feed twice must leave the target unchanged. How would you choose the UPSERT match key and update rule and handle duplicate source rows?

Answer:

Match key must be a unique natural key (e.g., `SKU`). The update rule updates non-key attributes only when values change. Deduplicate staging source rows using `DISTINCT ON (SKU)` prior to execution to enforce idempotency.

82. The subquery may return NULL. Why can NOT IN and NOT EXISTS give different results when finding customers without orders?

Answer:

If subquery contains a single `NULL`, `NOT IN` evaluates to `UNKNOWN` for all rows and returns 0 results. `NOT EXISTS` uses boolean row matching unaffected by `NULL` presence.

83. An inventory query returns new matching rows when repeated in the same transaction. Distinguish phantom reads from non-repeatable reads and explain the role of isolation.

Answer:

- **Non-Repeatable Read:** Existing row updated/deleted by concurrent transaction.
- **Phantom Read:** New rows inserted by concurrent transaction. Mitigated by setting transaction isolation to `SERIALIZABLE`.

84. Applying a function to indexed OrderDate to filter by year yields a full scan. Describe an equivalent predicate and how to verify that it returns the same results.

Answer:

- **Non-Sargable:** `WHERE EXTRACT(YEAR FROM OrderDate) = 2026`
- **Equivalent Sargable Predicate:** `WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'`
- **Verification:** Run `EXPLAIN` to confirm Index Range Scan utilization.

85. A plan estimates 50 rows but actually returns 50,000. How could this affect join selection, and what would you check first?

Answer:

- **Join Selection Effect:** Optimizer chooses a Nested Loop join (optimal for 50 rows) instead of a Hash Join, causing catastrophic execution slowdowns.
- **First Action:** Check for stale statistics; execute `ANALYZE table_name;` to update optimizer statistics.

Practical Questions

86. Using Orders(CustomerID, OrderDate, Total), calculate each customer's total spending in one CTE and the average of those totals in another. Return customers above that average.

Answer:

```
WITH CustSpend AS (
    SELECT CustomerID, SUM(Total) AS TotalSpend FROM Orders GROUP BY CustomerID
),
AvgSpend AS (
    SELECT AVG(TotalSpend) AS OrgAvg FROM CustSpend
)
SELECT CustomerID, TotalSpend
FROM CustSpend, AvgSpend
WHERE TotalSpend > OrgAvg;
```

87. Use a recursive CTE on Employee (EmployeeID, ManagerID, Name) to list all subordinates of a given root and their depth. Provide three levels of test data.

Answer:

```
WITH RECURSIVE SubTree AS (
    SELECT EmployeeID, ManagerID, Name, 1 AS Depth
    FROM Employee WHERE EmployeeID = 1 -- Root
    UNION ALL
    SELECT e.EmployeeID, e.ManagerID, e.Name, s.Depth + 1
    FROM Employee e JOIN SubTree s ON e.ManagerID = s.EmployeeID
)
SELECT * FROM SubTree;
```

88. From Orders (OrderID, CustomerID, OrderDate, Total), show each customer's running total. Use OrderID to break ties on OrderDate.

Answer:

```
SELECT CustomerID, OrderID, OrderDate, Total,
       SUM(Total) OVER(
           PARTITION BY CustomerID
           ORDER BY OrderDate, OrderID
       ) AS RunningTotal
FROM Orders;
```

89. From Sales (Region, SellerID, Amount), aggregate sales per seller within each region and use DENSE_RANK to return the top two ranks. Test tied totals.

Answer:

```
WITH RankedSales AS (
    SELECT Region, SellerID, SUM(Amount) AS TotalSales,
           DENSE_RANK() OVER(PARTITION BY Region ORDER BY SUM(Amount) DESC) AS rnk
    FROM Sales GROUP BY Region, SellerID
)
SELECT * FROM RankedSales WHERE rnk <= 2;
```

90. Extract brand from the Details JSON column in Product (ProductID, Details). Build a JSON object containing ProductID and brand. Show the result when brand is missing.

Answer:

```
SELECT ProductID,
       json_build_object('ProductID', ProductID, 'Brand', Details->>'brand') AS Result
FROM Product;
-- Missing brand attributes return JSON object with 'Brand': null
```

91. Use correlated NOT EXISTS on Customers and Orders to find customers who did not order last month. Test orders immediately before and within the date range.

Answer:

```
SELECT c.CustomerID, c.Name
FROM Customer c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders o
    WHERE o.CustomerID = c.CustomerID
      AND o.OrderDate >= '2026-10-01' AND o.OrderDate < '2026-11-01'
);
```

92. Use GROUPING SETS or ROLLUP on Sales (Region, Category, Amount) to show region-category subtotals, region totals, and a grand total.

Answer:

```
SELECT Region, Category, SUM(Amount) AS TotalAmount
FROM Sales
GROUP BY ROLLUP(Region, Category);
```

93. Use LATERAL or APPLY to return the two latest orders for each customer. Include customers without orders and specify a tie-break rule.

Answer:

```
SELECT c.CustomerID, o.*
FROM Customer c
LEFT JOIN LATERAL (
    SELECT OrderID, OrderDate, Total
    FROM Orders
    WHERE CustomerID = c.CustomerID
    ORDER BY OrderDate DESC, OrderID DESC
    LIMIT 2
) o ON TRUE;
```

94. Pivot MonthlySales (Region, MonthNo, Amount) so months 1, 2, and 3 appear as separate columns. Show zero for missing months and verify the totals.

Answer:

```
SELECT Region,
       COALESCE(SUM(CASE WHEN MonthNo = 1 THEN Amount END), 0) AS Month_1,
       COALESCE(SUM(CASE WHEN MonthNo = 2 THEN Amount END), 0) AS Month_2,
       COALESCE(SUM(CASE WHEN MonthNo = 3 THEN Amount END), 0) AS Month_3
FROM MonthlySales
GROUP BY Region;
```

95. Write a SKU-based MERGE or UPSERT from StageProduct(SKU, Name, Price) into Product. Run the same input twice and verify that the target row count and values remain unchanged.

Answer:

```
INSERT INTO Product (SKU, Name, Price)
SELECT SKU, Name, Price FROM StageProduct
ON CONFLICT (SKU) DO UPDATE
SET Name = EXCLUDED.Name, Price = EXCLUDED.Price;
-- Duplicate executions keep row count unchanged.
```

96. Create an invoice within a transaction. If an optional discount update fails after a SAVEPOINT, roll back only that update and commit the invoice. Verify the final row.

Answer:

```
BEGIN;
INSERT INTO Invoice VALUES (5001, 250.00);
SAVEPOINT discount_stage;
UPDATE Invoice SET Total = Total - 50 WHERE InvoiceID = 99999; -- Fails
ROLLBACK TO SAVEPOINT discount_stage;
COMMIT; -- Invoice 5001 committed successfully
```

97. In a test database, use two client sessions to update two inventory rows in opposite orders and demonstrate a deadlock. Repeat using a consistent lock order and compare the results.

Answer:

- **Opposite Order:** Session 1 updates Row A then B; Session 2 updates Row B then A. Causes Deadlock error.
- **Consistent Order:** Both sessions update Row A then Row B sequentially. Session 2 blocks safely without deadlocking.

98. On a large Orders table, run a query with a function on the date column and an equivalent date-range query. Verify equal results and compare their access paths using EXPLAIN.

Answer:

```
EXPLAIN ANALYZE SELECT * FROM Orders WHERE DATE(OrderDate) = '2026-01-01'; -- Seq Scan
EXPLAIN ANALYZE SELECT * FROM Orders WHERE OrderDate >= '2026-01-01' AND OrderDate <
'2026-01-02'; -- Index Scan
```

99. As part of a data correction, update Customer status and the flagged value of related Orders. Verify the results using an invalid ID, NULL, duplicate inputs, and a large sample.

Answer:

```
BEGIN;
UPDATE Customer SET Status = 'Suspended' WHERE CustomerID = 10;
UPDATE Orders SET IsFlagged = TRUE WHERE CustomerID = 10;
-- Test Edge cases (NULL/invalid IDs yield 0 rows updated)
COMMIT;
```

100. Compare execution plans for an Orders query filtered by CustomerID and OrderDate before and after adding a composite index. Document estimated and actual rows, result equivalence, and the script change log.

Answer:

- **Before Index:** Execution Plan: Sequential Scan | Estimated Rows: 50 | Actual Execution Time: 450ms.
- **After Index:** Execution Plan: Index Scan using `idx_cust_date` | Actual Execution Time: 2ms. Results identical.

Module 5: Database Security and Routine Maintenance

Theory Questions

101. Define separate roles for a reporting user, a data-entry user, and an administrator. Explain which permissions each needs under least privilege.

Answer:

- **Reporting User:** `SELECT` permission on specific reporting views/tables only.
- **Data-Entry User:** `SELECT`, `INSERT`, `UPDATE` on operational transaction tables; no DDL or `DELETE` rights.
- **Administrator:** `SUPERUSER` / `ALL PRIVILEGES` for schema modification, role management, and maintenance.

102. Many failed logins come from an unfamiliar IP address. Explain the difference between authentication and authorization and the role of account lockout.

Answer:

- **Authentication:** Verifies *who* the user is (credentials verification).
- **Authorization:** Verifies *what* an authenticated user is permitted to do.
- **Account Lockout:** Temporarily disables credentials after N failed attempts to prevent automated brute-force attacks.

103. Customer data appears in database files, network traffic, and backups. Which form of encryption applies to each?

Answer:

- **Database Files:** Transparent Data Encryption (TDE) / At-Rest Encryption (AES-256).
- **Network Traffic:** TLS / SSL Encryption In-Transit.
- **Backups:** Encrypted Backup Files (Asymmetric / Symmetric key backup encryption).

104. A support report should reveal only the last four digits of a phone number. Explain how masking meets this need and who should be able to see the full number.

Answer:

Dynamic Data Masking obfuscates plain text (e.g., `XXX-XXX-1234`) at query execution for non-privileged support roles. Full unmasked access is restricted exclusively to authorized Compliance/DPO personnel.

105. A database fails to restart after shutdown. How would you use the error log, configuration file, and available disk space to isolate the cause?

Answer:

1. Inspect `error.log` for fatal crash codes or startup panic messages.
2. Check `postgresql.conf` / `my.cnf` for invalid parameter syntax or port conflicts.
3. Execute `df -h` to check if disk volume is at 100% capacity, preventing lockfile creation.

106. A user retains excess access after a role change. In what order would you investigate direct grants, inherited roles, and default permissions?

Answer:

1. **Direct Grants:** Check permissions explicitly assigned to user account (`information_schema.table_privileges`).
2. **Inherited Roles:** Check active group/role memberships assigned to the user.
3. **Default Permissions:** Inspect global/schema-level default permissions granted to `PUBLIC`.

107. A login form places user input directly into SQL. Explain how SQL injection could occur and how parameter binding prevents it.

Answer:

- **SQL Injection:** Attacker inputs `' OR '1'='1`, altering query logic structure to bypass login checks.
- **Parameter Binding:** Treats input strictly as literal parameter data values rather than executable SQL code tokens.

108. A suspicious export runs during business hours. Explain the order of containment, escalation, preservation of audit evidence, and incident documentation.

Answer:

1. **Containment:** Terminate suspicious DB session and lock account immediately.
2. **Escalation:** Notify CISO and Security Operations Center (SOC).
3. **Preservation:** Secure transaction logs, audit traces, and connection history.
4. **Documentation:** Complete formal Security Incident Report detailing timeline and exposure.

109. Storage usage is at 85%, the log is growing quickly, and the backup job failed. Which evidence would you check first, and at what threshold would you take urgent action?

Answer:

- **First Check:** Backup job error log to determine why transaction logs are not truncating.
- **Urgent Threshold:** Take immediate action at **85%** (trigger manual log truncation/volume expansion) before disk hits 100% and triggers database crash/read-only lock.

110. Available RAM is limited as application workload increases. Explain how you would allocate memory considering the database instance, active sessions, and the operating system.

Answer:

- **Operating System:** Reserve 15-20% of total system RAM.
- **DB Instance Buffer Pool:** Allocate 60-70% RAM for shared data caching.
- **Active Sessions:** Restrict per-session work memory (e.g., `work_mem`) to prevent OOM killer during concurrent joins.

Practical Questions

111. In a sandbox database, create analyst and clerk roles. Grant the analyst SELECT and the clerk the required INSERT permission. Test both allowed and denied statements.

Answer:

```
 CREATE ROLE analyst; CREATE ROLE clerk;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst;
GRANT INSERT ON Orders TO clerk;

-- Analyst Test:
SELECT * FROM Orders; -- Success
INSERT INTO Orders VALUES (...); -- Fails: Permission Denied
```

112. Revoke UPDATE from a test user. Reconnect as that user and document the results of SELECT and UPDATE separately.

Answer:

```
 REVOKE UPDATE ON Customer FROM test_user;
-- Test user session:
SELECT * FROM Customer; -- Succeeds
UPDATE Customer SET Name = 'New' WHERE CustomerID = 1; -- Error: permission denied for
table Customer
```

113. Use your DBMS's supported method to configure password policy and lockout for a test account. Lock an unused test account and verify the login outcome.

Answer:

```
ALTER USER test_user WITH PASSWORD 'SecurePass123!' VALID UNTIL '2027-01-01';
ALTER USER test_user ACCOUNT LOCK;
-- Connection Attempt outcome: "FATAL: account is locked".
```

114. Create a view that masks Phone in Customer(CustomerID, Name, Phone). Verify a restricted user's access to the view and to the base table separately.

Answer:

```
CREATE VIEW v_masked_customer AS
SELECT CustomerID, Name, 'XXXXXX' || RIGHT(Phone, 4) AS Phone FROM Customer;

GRANT SELECT ON v_masked_customer TO clerk;
-- Clerk query on view: Sees masked phone.
-- Clerk query on Customer table: Permission Denied.
```

115. Restrict access to a test database to an approved client IP or subnet. Test from an authorized and an unauthorized client.

Answer:

```
# pg_hba.conf configuration
host    app_db    app_user    192.168.1.50/32    scram-sha-256
host    app_db    app_user    0.0.0.0/0          reject
```

Outcome: Authorized IP 192.168.1.50 connects; external IPs rejected.

116. Use parameter binding in a SQL client to look up a row by CustomerID in a test database. Verify the query's behavior with unexpected input.

Answer:

```
PREPARE stmt_cust(int) AS SELECT * FROM Customer WHERE CustomerID = $1;
EXECUTE stmt_cust(101); -- Success
EXECUTE stmt_cust('101 OR 1=1'); -- Fails at type cast safely
```

117. Prepare a daily health check for a test instance. Record service, listener, disk usage, error logs, and the last backup job's status.

Answer:

Metric	Check Command / Source	Status
DB Service	systemctl status postgresql	OK (Active/Running)
Listener	pg_isready -h localhost	OK (Accepting connections)
Disk Usage	df -h /var/lib/postgresql	OK (42% used)
Error Log	tail -n 50 postgresql.log	OK (0 fatal errors)
Last Backup	pg_stat_archiver	OK (Success at 02:00)

118. Configure TLS on a test server and verify that the client session is encrypted. Require a secure connection and document what happens to an unencrypted connection.

Answer:

```
 -- Verification query:
SELECT ssl, version FROM pg_stat_ssl WHERE pid = pg_backend_pid();
-- Result: ssl = true (TLSv1.3). Non-TLS connection attempts are rejected.
```

119. Use a column encryption feature supported by your DBMS to protect a sample sensitive column. Show the outcomes for authorized and restricted users.

Answer:

```
 -- Encrypting data:
UPDATE Customer SET SSN = pgcrypto.pgp_sym_encrypt(SSN, 'SecretKey');
-- Authorized query (with key): Returns plain SSN.
-- Restricted query (without key): Returns unintelligible bytea string.
```

120. Enable login auditing and DDL or DML auditing for a sensitive table. Generate a test event and identify its user, action, timestamp, and outcome in the log.

Answer:

```
 LOG: STATEMENT: ALTER TABLE Customer ADD COLUMN SSN VARCHAR(11);
USER: admin_user | TIMESTAMP: 2026-11-09 10:15:22 UTC | ACTION: ALTER TABLE | OUTCOME:
SUCCESS
```

121. Assess patch level, default accounts, excessive privileges, and network exposure in a test database. Write a vulnerability report with severity and corrective action for each finding.

Answer:

Vulnerability	Severity	Corrective Action
Unpatched DB Engine (v15.1)	High	Apply minor patch update to v15.5 immediately.
Default 'postgres' password active	Critical	Alter password to complex string.
Public SELECT on HR schema	Medium	Revoke PUBLIC permissions on sensitive schemas.

122. Back up a test instance's configuration. Reproduce a startup failure using an incorrect port or parameter. Diagnose it from the error log, restore the configuration, and verify service recovery.

Answer:

1. Backup: `cp postgresql.conf postgresql.conf.bak`
2. Induce Failure: Set `port = 'INVALID'`; restart fails.
3. Diagnosis: Log states `"invalid integer for line 40: port"`.
4. Restore: `cp postgresql.conf.bak postgresql.conf`; DB restarts successfully.

123. Create a blocking scenario using two test sessions. Use a system view to identify the blocker and waiting session. Verify the query outcome after the lock is released and document it.

Answer:

```
-- Identify blocker:
SELECT blocking_pid, blocked_pid, current_query_in_blocking_process
FROM pg_stat_activity;
-- Session 1 COMMIT releases lock; Session 2 unblocks and completes update.
```

124. Prepare a six-month capacity worksheet. Current data is 120 GB, monthly data growth is 10 GB, log allocation is 30 GB, and backup allocation is 160 GB. Show data, log, backup, and additional headroom separately.

Answer:

- **6-Month Data Growth:** 120 GB + (6 × 10 GB) = 180 GB
- **Log Allocation:** 30 GB
- **Backup Allocation:** 160 GB
- **Headroom (20% safety margin):** 74 GB
- **Total Required Storage:** 444 GB

125. Schedule statistics collection, space reclamation, or a supported integrity check in a lab database. Document how to identify successful and failed runs in the runbook.

Answer:

```
# Cron job entry:
0 2 * * * postgres vacuumdb --all --analyze-in-jobs=2 >> /var/log/db_maintenance.log 2>&1
# Check outcome:
grep -E "VACUUM|ERROR" /var/log/db_maintenance.log
```

Module 6: Backup Recovery Upgrades and Cloud Databases

Theory Questions

126. A business can tolerate 15 minutes of data loss and two hours of downtime. Identify RPO and RTO and explain how they affect backup planning.

Answer:

- **RPO (Recovery Point Objective):** 15 minutes (max allowable data loss). Dictates WAL/log backup frequency at ≤ 15 min intervals.
- **RTO (Recovery Time Objective):** 2 hours (max allowable downtime). Dictates high-speed physical restoration tooling and automated failover strategy.

127. Compare daily logical and periodic physical backups. Which is suitable for restoring an individual table, and which suits full server recovery?

Answer:

- **Logical Backup (SQL Dump):** Slower, human-readable format. Ideal for restoring individual tables/schemas.
- **Physical Backup (File Block Snapshot):** Fast binary restore. Ideal for full server disaster recovery.

128. Explain the trade-offs between hot and cold backups for a 24-hour service. Include the role of transaction logs.

Answer:

- **Hot Backup:** Performed while DB is live. 0 downtime, but requires continuous transaction log archiving (WAL) to ensure point-in-time consistency.
- **Cold Backup:** Requires complete DB service shutdown. Ensures clean state without log replay, but causes major service availability disruption.

129. How do patching, backups, and access control responsibilities differ between a self-managed database on a cloud VM and a managed database service?

Answer:

- **Self-Managed Cloud VM:** Customer manages OS patching, DB engine installation, automated backup scripts, and network security.
- **Managed DB Service (RDS):** Cloud Provider manages OS/DB engine patching and automated backups; Customer manages schema, data access, and IAM policies.

130. Compare vertical scaling and a read replica for a read-heavy reporting workload. How can replica lag affect report results?

Answer:

- **Vertical Scaling:** Increases primary CPU/RAM; expensive, hard limits.
- **Read Replica:** Offloads SELECT queries across parallel instances.
- **Replica Lag Effect:** Replication delay may cause reporting queries to show stale/out-of-date data relative to primary.

131. An accidental DELETE occurs at noon. A full backup from 6 a.m. and continuous transaction logs are available. Compare the likely outcomes of complete and point-in-time recovery.

Answer:

- **Complete Recovery:** Replays all logs including the noon DELETE, losing the deleted data.
- **Point-In-Time Recovery (PITR):** Restores 6 a.m. backup and replays logs up to 11:59:59 AM, successfully recovering data right before deletion.

132. How do prerequisites, fallback, and post-change validation differ between a security patch within the same major release and an upgrade to a new major version?

Answer:

- **Minor Security Patch:** Low risk, fully backward compatible; in-place upgrade with simple binary rollback.
- **Major Version Upgrade:** Potential breaking syntax changes; requires full pg_dump/pg_upgrade compatibility checks, mandatory complete backup fallback, and extensive post-upgrade application integration testing.

133. Traffic doubles on several days each month, and data must stay in a specified region. Explain your cloud design choices considering region, scaling, compute, storage, and transfer costs.

Answer:

Deploy in specified local sovereign region (e.g., eu-central-1). Use Managed DB with Auto-Scaling Compute (or Read Replicas) to handle peak traffic bursts. Utilize auto-expanding storage to minimize fixed idle compute costs and keep traffic within same-AZ to avoid data egress fees.

134. A cloud diagram places the application and database in one availability zone with automated backups enabled. Assess service continuity and data recovery separately if the zone fails.

Answer:

- **Service Continuity:** FAILS (Total outage; no Multi-AZ failover standby available).
- **Data Recovery:** SUCCEEDS (Automated snapshot backups stored durably in regional cloud storage can be restored into a alternate active zone).

135. A proposed OLTP design adds a read replica and automatic failover. Which handles read load and which handles primary failure? Explain the effect on RPO and RTO.

Answer:

- **Read Replica:** Offloads read query load.
- **Automatic Failover (Standby):** Promotes to primary during node failure.
- **Metrics Impact:** Reduces RTO to seconds (automatic failover) and lowers RPO to near-zero (synchronous/near-synchronous replication).

Practical Questions

136. Take a logical backup of a test database using your DBMS's backup tool. Restore it to a separate database and compare row counts and results from two sample queries.

Answer:

```
 # Backup & Restore execution:
pg_dump -U postgres -d testdb > testdb_backup.sql
createdb -U postgres testdb_restored
psql -U postgres -d testdb_restored < testdb_backup.sql

# Verification:
psql -d testdb -c "SELECT COUNT(*) FROM Customer;"
psql -d testdb_restored -c "SELECT COUNT(*) FROM Customer;" -- Counts match
```

137. Schedule a daily test backup job. Specify its output, error log, and retention rules. Verify one successful run.

Answer:

```
 # Backup script (cron_backup.sh):
#!/bin/bash
pg_dump -U postgres app_db | gzip > /backups/app_db_$(date +%Y%m%d).sql.gz 2>>
/backups/err.log
find /backups/ -type f -mtime +7 -delete # 7-day retention rule
```

138. Draw a 3-2-1 plan from a sample backup inventory using two types of media and one offsite copy. State the approved sequence for verifying backups and removing old copies.

Answer:

- **3 Copies:** Primary DB, Local On-Site Disk Backup, Remote Cloud Backup.
- **2 Media Types:** Local NVMe Storage array + AWS S3 Cloud Object Storage.
- **1 Offsite Copy:** AWS S3 Cloud Storage (Different geographical region).
- **Sequence:** 1. Verify restore checksum -> 2. Execute automated spin-up test -> 3. Purge expired backups per retention policy.

139. Compare two managed cloud database services for a specified workload. Tabulate engine, compute, storage, availability, backup, and estimated cost assumptions.

Answer:

Feature	AWS RDS PostgreSQL	Azure Database for PostgreSQL
Engine	PostgreSQL 16	PostgreSQL 16
Compute	db.m6g.xlarge (4 vCPU / 16GB)	Standard_D4ds_v5 (4 vCPU / 16GB)
Storage	gp3 Auto-scaling IOPS	Premium SSD v2
Availability	Multi-AZ Standby (99.95%)	Zone-Redundant HA (99.95%)
Est. Monthly Cost	~$350 / month	~$360 / month

140. Create a managed database in an approved sandbox. Choose an engine, region, storage, and network configuration. Verify a connection from an authorized client.

Answer:

```
 # AWS CLI Deployment example:
aws rds create-db-instance \
  --db-instance-identifier sandbox-db \
  --db-instance-class db.t4g.micro \
  --engine postgres \
  --allocated-storage 20 \
  --master-username adminuser --master-user-password Password123!

# Connection Test:
psql -h sandbox-db.c123.us-east-1.rds.amazonaws.com -U adminuser -d postgres
```

141. Create Customer and Orders tables in a cloud test database. Add primary and foreign keys, execute INSERT, UPDATE, and SELECT, and record the results.

Answer:

```
 CREATE TABLE Customer (CustomerID INT PRIMARY KEY, Name VARCHAR(50));
CREATE TABLE Orders (OrderID INT PRIMARY KEY, CustomerID INT REFERENCES
Customer(CustomerID), Total DECIMAL(10,2));

INSERT INTO Customer VALUES (1, 'Cloud User');
INSERT INTO Orders VALUES (1001, 1, 99.99);
UPDATE Orders SET Total = 120.00 WHERE OrderID = 1001;
SELECT * FROM Orders; -- Success: Returns Order 1001 with Total 120.00
```

142. Review CPU, connections, and storage metrics in a cloud sandbox. Configure a storage alert and record its threshold and notification test result.

Answer:

- **Alert Target:** FreeStorageSpace < 5 GB.
- **Action Group:** Send SNS Email notification to DBA team.
- **Test Outcome:** Triggered artificial disk fill test; CloudWatch alarm transitioned to `ALARM` state and email received within 2 minutes.

143. After backing up a test database, make two valid changes and perform an accidental DELETE. Use point-in-time recovery to restore to just before the DELETE. Verify that the valid changes and deleted rows are present.

Answer:

1. Backup taken at 10:00. Valid Insert A (10:05), Valid Insert B (10:10). Accidental DELETE (10:15).
2. Initiate PITR targeting timestamp `10:14:59`.
3. Restored instance contains Inserts A & B alongside all rows preserved prior to DELETE.

144. Check prerequisites and conflicts for a selected patch in a test environment. Keep a verified fallback backup, apply the patch, and validate version, startup, and sample queries.

Answer:

```
 # Apply minor version update:
sudo apt-get update && sudo apt-get --only-upgrade install postgresql-15

# Verification:
psql -U postgres -c "SELECT version();" -- Output: PostgreSQL 15.5
# Run sample validation query:
SELECT COUNT(*) FROM Orders; -- Success
```

145. Write and execute an out-of-place upgrade runbook in a lab. Show prechecks, data transfer, application validation, the downtime window, and the fallback point.

Answer:

1. **Downtime Window:** 02:00 - 04:00 AM UTC.
2. **Precheck:** Validate extensions compatibility on target DB v16 instance.
3. **Data Transfer:** `pg_dumpall | psql -h target-v16-db`
4. **Validation:** Point staging app to Target DB; run integration tests.
5. **Fallback Point:** Re-route DNS back to legacy v15 instance if errors occur.

146. Restore an automated cloud database backup or snapshot into a new test instance. Verify consistency and access settings, and record the measured recovery time.

Answer:

```
aws rds restore-db-instance-from-db-snapshot \
  --target-db-instance-identifier restored-test-db \
  --db-snapshot-identifier rds-snapshot-2026-11-01
```

Measured Recovery Time (RTO): 8 minutes 42 seconds. Data consistency verified.

147. Draw a cloud OLTP architecture showing a private subnet, primary, standby or replica, application access, encryption, and automated backup. Label the data flows.

Answer:

```
[ Public Subnet ]  ---> App Server (TLS Encrypted Connection)
                            |
                            v  (Private Subnet)
[ Private Subnet ] ---> [ Primary DB ] ===== (Sync Repl) =====> [ Standby DB ]
                            |
                            +---> (Automated Encrypted Backups) ---> [ AWS S3 ]
```

148. A cloud design has a public database endpoint, a shared admin account, and an unencrypted connection. Redesign it and show where IAM, private access, TLS, and audit logging fit.

Answer:

- **Private Access:** Move DB endpoint to Private Subnet (no public IP); access via VPC Peering / VPN only.
- **IAM Integration:** Replace shared admin password with IAM Database Authentication.
- **Encryption & Audit:** Enforce `rds.force_ssl = 1` (TLS) and enable CloudWatch DB Audit Logging.

149. Write a disaster recovery runbook for a region outage using cross-region backups. Include restore order, application reconnection, data validation, and measurement of RPO and RTO.

Answer:

1. **Declare DR Event:** Region A failure confirmed by Cloud Provider.
2. **Restore Order:** Spin up DB instance in Region B using latest cross-region replicated snapshot.
3. **App Reconnection:** Update Route53 DNS endpoints to point to Region B DB target.
4. **Validation:** Run automated smoke tests to confirm data write capability.
5. **Metrics:** RPO measured at 12 mins data loss; RTO measured at 38 mins total outage.

150. Read or create a sample Docker Compose file for a database container. Identify the persistent volume, connection settings, and handling of secrets. Document whether you would use a container or a managed database for the same workload.

Answer:

```
version: '3.8'
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password # Handling secrets
    volumes:
      - pgdata:/var/lib/postgresql/data # Persistent volume
    ports:
      - "5432:5432"
volumes:
  pgdata:
```

Recommendation: Use Docker container for local dev/testing labs due to speed and portability; use Managed Cloud DB (e.g., AWS RDS) for production workloads requiring high availability and automated failover.